2520030117
skill-08

EXPERIMENT – 8

Title:

Variable Expansion and Built-in Command Handling in Custom Linux Shell

Aim:

To detect and expand variable references, handle undefined variables, update tokens, support nested variables, and execute built-in commands using a dispatch table.

Objective:

- To detect variable references.
- To expand variable values.
- To handle undefined variables.
- To update command tokens.
- To support nested variables.
- To test variable expansion.
- To identify built-in commands.
- To execute built-in commands.
- To handle invalid commands.
- To maintain shell state.

Theory:

A shell uses **variables** to store values that can be used while executing commands.

For example:

echo $HOME

expands `$HOME` to its stored value.

If a variable is not defined, the shell should handle it without crashing.

A shell also has **built-in commands** such as `cd` and `exit`. These commands can be executed directly by the shell without creating a separate child process.

A **dispatch table** can be used to identify a command and call its corresponding built-in function.

Algorithm:

1. Start the custom shell.
2. Read the command from the user.

3. Detect variable references such as `$HOME`.
4. Find the value of the variable.
5. Replace the variable with its value.
6. Handle undefined variables.
7. Update the command tokens.
8. Check whether the command is a built-in command.
9. Use the dispatch table to identify the built-in. 10. Execute the built-in command.
11. Handle invalid commands.
12. Maintain the shell state.
13. Repeat until `exit` is entered.
14. Stop.

Functions Used:

Function	Purpose
`getenv( )`	commands Split
`strcmp( )`	command into
`strtok( )`	tokens Display
`printf( )`	Get variable value
	output
Compare	
`chdir()`	Change directory
`getcwd( )`	Get current directory
`exit()`	Exit the shell

Sample Input:

2520030117_vaishali> echo $HOME 2520030117_vaishali> cd ..
2520030117_vaishali> pwd
2520030117_vaishali> xyzabc

Sample Output:

/home/vaishali
/home

Command not found

Observation:

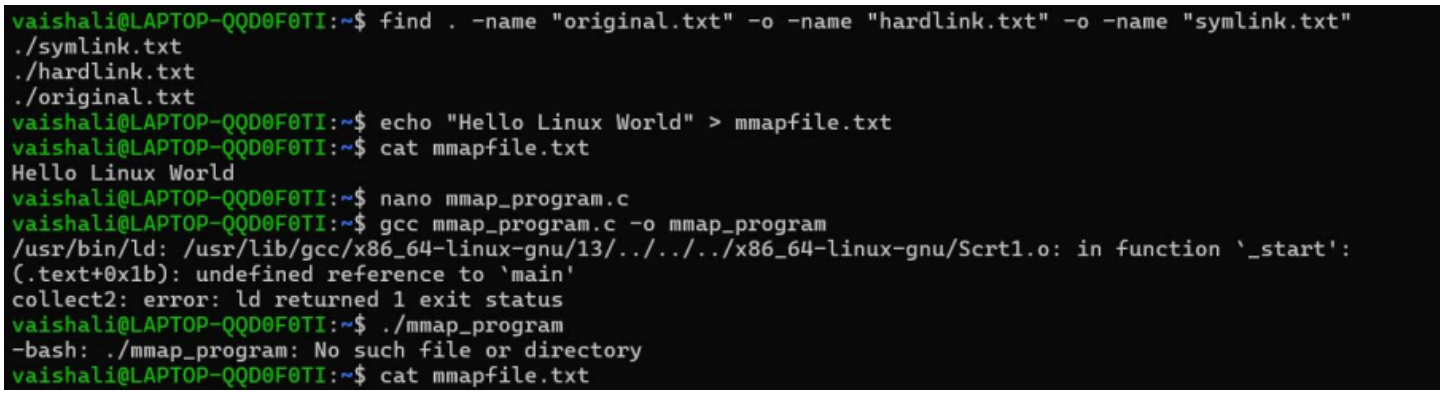
1. Variable references were detected.
2. Variable values were expanded.
3. Undefined variables were handled.
4. Command tokens were updated.
5. Built-in commands were identified.
6. Built-in commands were executed inside the shell.
7. Invalid commands were handled.
8. Shell state was maintained.

Result:

The custom shell successfully handled **variable expansion and built-in command dispatch**.

Conclusion:

The experiment successfully demonstrated variable expansion, handling of undefined variables, built-in command execution, invalid command handling, and shell state management.



```
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/us
r/games:/usr/local/games:/usr/lib/wsl/lib:/mnt/c/Program Files/C
ommon Files/Oracle/Java/javapath:/mnt/c/Program Files (x86)/Comm
on Files/Oracle/Java/java8path:/mnt/c/Program Files (x86)/Common
 Files/Oracle/Java/javapath:/mnt/c/Windows/system32:/mnt/c/Windo
ws:/mnt/c/Windows/System32/Wbem:/mnt/c/Windows/System32/WindowsP
owerShell/v1.0/:/mnt/c/Windows/System32/OpenSSH/:/mnt/c/Program
Files/dotnet/:/mnt/c/Program Files/Git/cmd:/mnt/c/ProgramData/ch
ocolatey/bin:/mnt/c/Program Files/nodejs/:/mnt/c/Users/kruthi/Ap
pData/Local/Programs/Python/Python310/Scripts/:/mnt/c/Users/krut
hi/AppData/Local/Programs/Python/Python310/:/mnt/c/Users/kruthi/
.local/bin:/mnt/c/Users/kruthi/AppData/Local/Microsoft/WindowsAp
ps:/mnt/c/Program Files/Java/jdk-25/bin:/mnt/c/Users/kruthi/AppD
ata/Local/Programs/Microsoft VS Code/bin:/mnt/c/Users/kruthi/App
Data/Local/GitHubDesktop/bin:/mnt/c/Users/kruthi/AppData/Roaming
/npm:/mnt/c/Users/kruthi/AppData/Local/Kingsoft/WPS Office/12.1.
0.27458/clitool:/snap/bin
2520030340_kruthi> echo $ABCXYZ
Undefined variable: ABCXYZ
```